

Relatório do Trabalho Prático 2 - MAC0352

Sistema de Gerenciamento de Rede em C sobre TCP

Nomes:

Laufernando Souza Dias - 11222947
Rafael Lopes Santana - 14604420
João Pedro de Toledo - 10724177
Erick Henrique Imai Rodrigues - 7577880

1. Arquitetura

O sistema consiste em uma aplicação de monitoramento de rede inspirada no protocolo SNMP (Simple Network Management Protocol), implementada em C com suporte a POSIX. A aplicação é composta por dois tipos de processo independentes — agente e manager — que se comunicam via TCP usando um protocolo de mensagens em texto ASCII.

O **agente** é responsável por expor métricas do host em que está sendo executado. Ao iniciar, ele abre um socket TCP em uma porta configurável e aguarda conexões do manager. Para cada conexão recebida, uma thread dedicada é criada para processar as requisições. O agente consulta o sistema de arquivos virtual /proc do Linux para coletar os dados, respondendo a cada GET com o valor atual da métrica solicitada. Os coletores são injetados em tempo de execução na tabela MIB via `mib_set_resolver`, separando a lógica de coleta da estrutura da MIB.

O **manager**, por sua vez, é o processo central de gerenciamento. Ele lê um arquivo de configuração (`agents.conf`) que lista os agentes conhecidos — cada um identificado por um nome, endereço IP e porta. Para cada agente configurado, o manager mantém uma thread de coleta (`worker`) que periodicamente abre uma conexão TCP, envia requisições GET para todos os OIDs listados na MIB, recebe as respostas, registra os valores em um arquivo CSV de histórico e atualiza o estado interno do agente (UP ou DOWN). O manager também renderiza uma tabela no terminal, atualizada a cada segundo por padrão, exibindo o estado em tempo real de todos os agentes monitorados.

Ademais, o protocolo TCP é utilizado na comunicação entre manager e agentes por oferecer entrega ordenada e confiável de bytes, simplificando o tratamento de mensagens no nível da aplicação. A conexão é aberta pelo manager a cada ciclo de coleta e encerrada após o recebimento de todas as respostas — modelo `connect-per-cycle` — o que torna qualquer falha de conexão ou `recv` diretamente detectável como evidência de indisponibilidade do agente.

2. Protocolo de comunicação

A unidade de troca de dados entre manager e agente é chamada de PDU (Protocol Data Unit). O formato adotado é texto ASCII delimitado por `\n`, com campos separados pelo caractere `|`:

VERSION|MSG_ID|TYPE|OID|VALUE\n

Campo	Descrição
VERSION	Versão do protocolo (atualmente 1); reservado para compatibilidade futura
MSG_ID	Inteiro de 32 bits que correlaciona cada requisição à sua resposta
TYPE	Tipo da mensagem: GET, RESPONSE, ERROR ou TRAP (reservado)
OID	Identificador da métrica solicitada (ex.: 1.1.1)
VALUE	Vazio em GET; valor da métrica em RESPONSE; código em ERROR

Os caracteres `|` e `\n` são reservados e não podem aparecer em nenhum campo. O codec, implementado em `src/protocol.c`, valida essa restrição tanto na codificação quanto na decodificação.

A escolha por texto ASCII em vez de um formato binário foi deliberada: facilita a depuração direta.

O MSG_ID cumpre o papel de camada de transporte simplificada: permite que o manager correlacione cada resposta à sua requisição correspondente. Como TCP já garante ordenação no stream, o `\n` é suficiente para delimitar PDUs sem necessidade de campos de comprimento.

Os códigos de erro definidos são:

Código	Slug	Significado
0	OK	Sem erro (não usado em mensagens ERROR)
1	NU_SUCH_ID	OID não pertence à MIB
2	BAD_REQUEST	PDU malformado ou tipo inesperado
3	INTERNAL	Falha ao ler a fonte do dado

4	UNSUPPORTED	Operação não suportada pelo agente
---	-------------	------------------------------------

3. Armazenamento dos Recursos (MIB)

A MIB é uma tabela estática compartilhada por agente e manager:

OID	Métrica	Fonte
1.1.1	CPU (%)	/proc/stat
1.1.2	Memória (%)	/proc/meminfo
1.1.3	Uptime (s)	/proc/uptime
1.2.1	Bytes in	/proc/net/dev
1.2.2	Bytes out	/proc/net/dev
1.3.1	Conexões TCP ativas	/proc/net/tcp{,6}

O acesso ao /proc é exclusivamente de leitura — trata-se de um sistema de arquivos virtual mantido pelo kernel na memória RAM, que expõe informações do sistema de forma segura. Ferramentas como top, htop e free utilizam exatamente as mesmas fontes.

Os valores coletados pelo manager são persistidos em history.csv, com uma linha por amostra:

```
timestamp,agent,oid,value
1777420519,node-a,1.1.1,11.99
```

O campo timestamp é epoch UNIX. Novas amostras são sempre anexadas ao final do arquivo (*append*), garantindo que o histórico completo seja preservado entre execuções.

4. Interfaces Entre Camadas

O projeto organiza a comunicação entre suas camadas por meio de interfaces bem definidas, evitando acoplamento direto entre os módulos.

A camada de aplicação interage com a camada de transporte/rede exclusivamente através das funções pdu_encode() e pdu_decode(), definidas em src/protocol.c. O

agente e o manager nunca manipulam bytes de rede diretamente — eles operam sobre structs pdu_t, e o codec é responsável por serializar e desserializar o formato VERSION|MSG_ID|TYPE|OID|VALUE\n.

A camada de MIB expõe uma única função de consulta, mib_get(oid, buf, bufsz), que retorna o valor atual da métrica identificada pelo OID ou um código de erro. O agente invoca essa função dentro do handler de requisições sem conhecer como o dado é coletado. Os coletores de /proc são registrados em tempo de execução via mib_set_resolver, permitindo que a tabela estática da MIB seja compartilhada entre agente e manager sem que nenhum dos dois precise conhecer os detalhes de leitura do sistema de arquivos.

A camada de enlace é simulada em software: erros de transmissão (como perda de pacotes) são representados por falhas de connect, timeouts de socket (SO_RCVTIMEO/SO_SNDTIMEO) ou respostas truncadas. Essa simulação é transparente para as camadas superiores — o scheduler do manager simplesmente contabiliza qualquer erro como falha de entrega, sem distinguir sua causa.

A interface entre scheduler e storage é igualmente isolada: o scheduler chama funções de storage.c para persistir amostras, sem conhecer o formato do CSV ou os detalhes de sincronização. O mutex global de escrita é encapsulado dentro de storage.c, invisível para o restante do sistema.

5. Decisões de Projeto e Concorrência

O sistema utiliza threads POSIX (pthread) em ambos os processos.

No agente, uma thread é criada para cada conexão TCP aceita (pthread_create seguido de pthread_detach). Essa thread lê PDUs do socket até o encerramento da conexão, processa cada requisição e envia a resposta, sem bloquear o loop principal de accept. Isso permite que múltiplos managers — ou múltiplos ciclos do mesmo manager — se conectem simultaneamente.

No manager, o scheduler mantém uma thread de coleta por agente configurado. Cada thread opera de forma independente: conecta ao seu agente, itera pelos OIDs, registra os resultados e dorme pelo período --interval. O acesso ao arquivo CSV é serializado por um mutex global em storage.c, evitando condições de corrida na escrita. O estado de cada agente (agent_t) também é protegido por um mutex individual, acessado tanto pela thread de coleta quanto pela thread de renderização da TUI.

6. Experimentos e Metodologia

Os experimentos realizados estão no diretório experiments/ no repositório da tarefa. Realizamos 4 conjuntos de teste, cujos resultados são gravados em experiments/results/.

6.1 Teste de Falha

Objetivo: medir o tempo entre a queda de um agente e a transição UP → DOWN registrada pelo manager.

Metodologia: o script run_failure.sh sobe um agente e um manager em loopback, aguarda o primeiro UP estabilizar, mata o agente com SIGTERM e mede, em nanossegundos, o intervalo até o manager registrar estado DOWN no log. O experimento é repetido 3 vezes.

Parâmetros: interval=1s, timeout=500ms, threshold=3.

TrialTempo até DOWN: 1: 3,057 s 2: 2,953 s 3: 2,953 s

Análise: o tempo teórico mínimo é $\text{threshold} \times \text{timeout} = 3 \times 500\text{ms} = 1,5\text{s}$ e o máximo é $\text{threshold} \times (\text{timeout} + \text{interval}) = 4,5\text{s}$. Os resultados (~3s) situam-se próximos ao ponto médio, com variância inferior a 110ms entre trials, indicando um scheduler de ritmo regular. O resultado confirma que a detecção é previsível e diretamente configurável pelos parâmetros --threshold, --timeout-ms e --interval.

6.2 Teste de Latência

Objetivo: medir o tempo de ida e volta (RTT) de um par GET/RESPONSE em função do número de requisições sequenciais enviadas.

Metodologia: a ferramenta bin/latency abre uma única conexão TCP com o agente e envia N GETs sequenciais para um OID, medindo o tempo de cada par em microssegundos. São calculados os percentis p50, p95 e a média.

N	p50(μs)	p95(μs)	média(μs)
1	342	342	342.0
10	128	213	135.0

100	132	183	136.9
1000	130	178	138.7

Análise: o valor elevado para $N=1$ reflete o custo de cache frio — primeira leitura de /proc, alocações iniciais do sistema operacional e aquecimento do scheduler. A partir de $N=10$, o sistema converge para $\sim 130\mu s$ por par GET/RESPONSE em loopback, dominado pelo custo de processamento do socket e leitura do /proc. A estabilidade dos resultados para $N \geq 10$ indica ausência de degradação com o aumento do volume de requisições.

6.3 Teste de Sobrecarga

Objetivo: quantificar o overhead de bytes introduzido pelas camadas do protocolo em relação ao dado útil transportado.

Metodologia: o script run_overhead.sh captura o tráfego na interface de loopback via tcpdump durante uma sessão de coleta e compara os bytes totais transmitidos com o tamanho dos valores de dados (campo VALUE das PDUs).

Análise: o formato VERSION|MSG_ID|TYPE|OID|VALUE\n introduz overhead fixo por PDU referente a cabeçalhos e delimitadores. Como os valores coletados são numéricos curtos (tipicamente 2–10 caracteres), o overhead relativo é expressivo em termos percentuais, mas irrelevante em termos absolutos de banda. O protocolo prioriza legibilidade e facilidade de depuração sobre eficiência de transmissão — uma troca adequada para o escopo do trabalho.

6.4 Teste de Escalabilidade

Objetivo: avaliar o impacto no consumo de CPU do manager à medida que o número de agentes monitorados aumenta.

Metodologia: o script run_scalability.sh sobe K agentes simultaneamente ($K = 1, 5, 10, 20$) e mede a CPU consumida pelo processo manager durante 4 segundos de coleta com interval=1s.

K agentes	CPU manager (%)	amostras
-----------	-----------------------	----------

1	0.00	30
5	0.25	150
10	0.75	300
20	1.75	600

Análise: o crescimento é aproximadamente linear, com incremento de ~0,09% de CPU por agente adicionado. O overhead por thread é baixo porque cada worker passa a maior parte do tempo bloqueado em I/O de rede, sem consumir CPU ativamente. O modelo de uma thread por agente se mostra viável para dezenas de nós monitorados sem impacto significativo no host do manager.

7. Limitações

Funcionalidades não implementadas

TRAPs: o tipo de PDU TRAP está reservado no protocolo, mas não implementado. O modelo atual é puramente reativo — o manager sempre inicia as requisições e o agente nunca notifica eventos assíncronos, como uso de CPU acima de um limiar ou queda de interface de rede. A ausência de TRAPs implica que eventos críticos só são descobertos no próximo ciclo de coleta, introduzindo latência de detecção desnecessária em casos que poderiam ser notificados imediatamente.

Autenticação e criptografia: as mensagens trafegam em texto puro, sem qualquer mecanismo de autenticação ou confidencialidade. Qualquer processo com acesso à rede pode enviar PDUs forjadas ao agente ou interceptar respostas do manager.

Agregação em memória: o histórico de amostras é mantido apenas no CSV. Não há estrutura em memória para calcular médias móveis, tendências ou disparar alertas em tempo real com base no histórico acumulado.

Limitações do protocolo

Modelo connect-per-cycle: o manager abre e fecha uma conexão TCP por ciclo de coleta por agente. Isso simplifica a detecção de falhas — qualquer erro de conexão é evidência de queda —, mas introduz overhead de handshake TCP a cada ciclo.

Uma conexão persistente reduziria a latência e o consumo de recursos, ao custo de maior complexidade no tratamento de reconexão e timeout.

Baseline de CPU: o cálculo de uso de CPU requer duas amostras de /proc/stat para computar um delta. O baseline é reiniciado a cada nova instância do processo agente, podendo gerar leituras imprecisas na primeira coleta após uma reinicialização.

Sem suporte a IPv6: o agente aceita conexões apenas em AF_INET (IPv4). O monitoramento de conexões TCP via /proc/net/tcp6 já está implementado na MIB, mas o socket de escuta não suporta clientes IPv6.

Limitações dos testes

Ambiente controlado (loopback): todos os experimentos foram realizados em loopback (127.0.0.1), onde a latência de rede é mínima e não há perdas de pacotes. Os resultados de latência e overhead não refletem condições reais de rede com jitter, perda de pacotes e largura de banda limitada.

Máquina única: agente e manager competem pelos mesmos recursos de CPU e memória durante os testes, o que pode inflar levemente as medições de consumo do manager, especialmente nos testes de escalabilidade com K=20.

Número reduzido de trials: os testes de falha utilizam apenas 3 repetições, insuficientes para análise estatística robusta. Um número maior de trials reduziria a influência de outliers e permitiria calcular intervalos de confiança confiáveis.

8. Referências

- J. F. Kurose, K. W. Ross, Computer Networking, A Top-Down Approach Featuring the Internet (6th revised edition), Addison-Wesley, 2012.
- FOROUZAN, Behrouz A. Comunicação de dados e redes de computadores. 4. ed. São Paulo: McGraw-Hill, 2008.
- <https://man7.org/linux/man-pages/man2/socket.2.html>
- https://www.php.net/manual/pt_BR/function.inet-ntop.php